

Style_Transfer_Exercise

March 20, 2026

Hugo Tanaka CSCI 611 Professor Bo Shen 2026 March 20

1 Style Transfer with Deep Neural Networks

In this notebook, we'll *recreate* a style transfer method that is outlined in the paper, [Image Style Transfer Using Convolutional Neural Networks](#), by Gatys in PyTorch.

In this notebook, we'll use a pre-trained VGG19 Net to extract content or style features from a passed in image. We'll then formalize the idea of content and style *losses* and use those to iteratively update our target image until we get a result that we want. You are encouraged to use a style and content image of your own.

```
[1]: # import resources
%matplotlib inline

from PIL import Image
from io import BytesIO
import matplotlib.pyplot as plt
import numpy as np

import torch
import torch.optim as optim
import requests
from torchvision import transforms, models
```

1.1 Load in VGG19 (features)

VGG19 is split into two portions: * `vgg19.features`, which are all the convolutional and pooling layers * `vgg19.classifier`, which are the three linear, classifier layers at the end

We only need the `features` portion, which we're going to load in and "freeze" the weights of, below.

```
[2]: # get the "features" portion of VGG19 (we will not need the "classifier"
↳portion)
vgg = models.vgg19(pretrained=True).features

# freeze all VGG parameters since we're only optimizing the target image
for param in vgg.parameters():
    param.requires_grad_(False)
```

```
/home/kanae/school/chicoSpring2026/CSCI611/Assignment4/as4/lib/python3.14/site-  
packages/torchvision/models/_utils.py:208: UserWarning: The parameter  
'pretrained' is deprecated since 0.13 and may be removed in the future, please  
use 'weights' instead.
```

```
warnings.warn(  
/home/kanae/school/chicoSpring2026/CSCI611/Assignment4/as4/lib/python3.14/site-  
packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a  
weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed  
in the future. The current behavior is equivalent to passing  
`weights=VGG19_Weights.IMAGENET1K_V1`. You can also use  
`weights=VGG19_Weights.DEFAULT` to get the most up-to-date weights.  
warnings.warn(msg)
```

```
[3]: # move the model to GPU, if available  
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
  
vgg.to(device)
```

```
[3]: Sequential(  
  (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (1): ReLU(inplace=True)  
  (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (3): ReLU(inplace=True)  
  (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,  
ceil_mode=False)  
  (5): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (6): ReLU(inplace=True)  
  (7): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (8): ReLU(inplace=True)  
  (9): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,  
ceil_mode=False)  
  (10): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (11): ReLU(inplace=True)  
  (12): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (13): ReLU(inplace=True)  
  (14): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (15): ReLU(inplace=True)  
  (16): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (17): ReLU(inplace=True)  
  (18): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,  
ceil_mode=False)  
  (19): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (20): ReLU(inplace=True)  
  (21): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (22): ReLU(inplace=True)  
  (23): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (24): ReLU(inplace=True)
```

```

(25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(26): ReLU(inplace=True)
(27): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
(28): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(29): ReLU(inplace=True)
(30): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(31): ReLU(inplace=True)
(32): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(33): ReLU(inplace=True)
(34): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(35): ReLU(inplace=True)
(36): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
)

```

1.1.1 Load in Content and Style Images

You can load in any images you want! Below, we've provided a helper function for loading in any type and size of image. The `load_image` function also converts images to normalized Tensors.

Additionally, it will be easier to have smaller images and to squish the content and style images so that they are of the same size.

```

[4]: def load_image(img_path, max_size=400, shape=None):
    ''' Load in and transform an image, making sure the image
        is <= 400 pixels in the x-y dims. '''
    if "http" in img_path:
        response = requests.get(img_path)
        image = Image.open(BytesIO(response.content)).convert('RGB')
    else:
        image = Image.open(img_path).convert('RGB')

    # large images will slow down processing
    if max(image.size) > max_size:
        size = max_size
    else:
        size = max(image.size)

    if shape is not None:
        size = shape

    in_transform = transforms.Compose([
        transforms.Resize(size),
        transforms.ToTensor(),
        transforms.Normalize((0.485, 0.456, 0.406),
                             (0.229, 0.224, 0.225))]

```

```

    # discard the transparent, alpha channel (that's the :3) and add the batch
    ↪dimension
    image = in_transform(image)[:3,:,:].unsqueeze(0)

    return image

```

Next, I'm loading in images by file name and forcing the style image to be the same size as the content image.

TODO: use your content vs style images.

Used photo of SF for content and Van Goth's drawing for style.

```

[5]: # load in content and style image
content = load_image('SF.jpg').to(device)
# Resize style to match content, makes code easier
style = load_image('goth.jpg', shape=content.shape[-2:]).to(device)

# helper function for un-normalizing an image
# and converting it from a Tensor image to a NumPy image for display
def im_convert(tensor):
    """ Display a tensor as an image. """

    image = tensor.to("cpu").clone().detach()
    image = image.numpy().squeeze()
    image = image.transpose(1,2,0)
    image = image * np.array((0.229, 0.224, 0.225)) + np.array((0.485, 0.456, 0.
    ↪406))
    image = image.clip(0, 1)

    return image

```

```

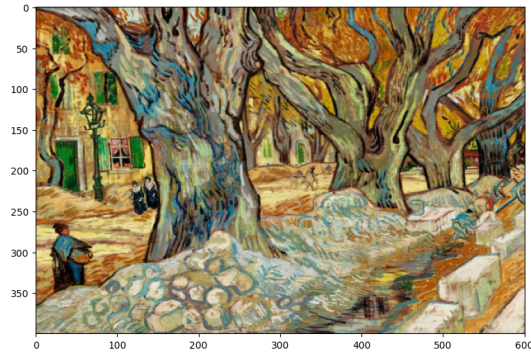
[6]: # display the images
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))
# content and style imgs side-by-side
ax1.imshow(im_convert(content))
ax2.imshow(im_convert(style))

```

```

[6]: <matplotlib.image.AxesImage at 0x7f3aca35ac10>

```



1.2 VGG19 Layers

To get the content and style representations of an image, we have to pass an image forward through the VGG19 network until we get to the desired layer(s) and then get the output from that layer.

```
[7]: # print out VGG19 structure so you can see the names of various layers
print(vgg)
```

```
Sequential(
  (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): ReLU(inplace=True)
  (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (3): ReLU(inplace=True)
  (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceiling_mode=False)
  (5): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (6): ReLU(inplace=True)
  (7): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (8): ReLU(inplace=True)
  (9): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceiling_mode=False)
  (10): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (11): ReLU(inplace=True)
  (12): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (13): ReLU(inplace=True)
  (14): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (15): ReLU(inplace=True)
  (16): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (17): ReLU(inplace=True)
  (18): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceiling_mode=False)
  (19): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (20): ReLU(inplace=True)
```

```

(21): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(22): ReLU(inplace=True)
(23): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(24): ReLU(inplace=True)
(25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(26): ReLU(inplace=True)
(27): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
(28): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(29): ReLU(inplace=True)
(30): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(31): ReLU(inplace=True)
(32): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(33): ReLU(inplace=True)
(34): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(35): ReLU(inplace=True)
(36): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
)

```

1.3 Content and Style Features

TODO: complete the mapping of layer names to the names found in the paper for the *content representation* and the *style representation*. The first layer (0) to conv1_1 has been done for you, below.

```

[8]: def get_features(image, model, layers=None):
    """ Run an image forward through a model and get the features for
        a set of layers. Default layers are for VGGNet matching Gatys et al.
        ↪ (2016)
    """

    ## TODO: Complete mapping layer names of PyTorch's VGGNet to names from the
    ↪ paper
    ## Need the layers for the content and style representations of an image
    if layers is None:
        layers = {
            '0': 'conv1_1',
            '5': 'conv2_1',
            '10': 'conv3_1',
            '19': 'conv4_1',
            '21': 'conv4_2',
            '28': 'conv5_1'
        }

    ## -- do not need to change the code below this line -- ##
    features = {}

```

```

x = image
# model._modules is a dictionary holding each module in the model
for name, layer in model._modules.items():
    x = layer(x)
    if name in layers:
        features[layers[name]] = x

return features

```

1.4 Gram Matrix

The output of every convolutional layer is a Tensor with dimensions associated with the `batch_size`, a depth, `d` and some height and width (`h`, `w`). The Gram matrix of a convolutional layer can be calculated as follows: * Get the depth, height, and width of a tensor using `batch_size`, `d`, `h`, `w` = `tensor.size()` * Reshape that tensor so that the spatial dimensions are flattened * Calculate the gram matrix by multiplying the reshaped tensor by it's transpose

Note: You can multiply two matrices using `torch.mm(matrix1, matrix2)`.

TODO: Complete the `gram_matrix` function.

```

[9]: def gram_matrix(tensor):
    """ Calculate the Gram Matrix of a given tensor
        Gram Matrix: https://en.wikipedia.org/wiki/Gramian\_matrix
    """

    ## get the batch_size, depth, height, and width of the Tensor
    ## reshape it, so we're multiplying the features for each channel
    ## calculate the gram matrix

    # https://docs.pytorch.org/docs/stable/generated/torch.Tensor.size.html
    batch_size, depth, height, width = tensor.size()

    # https://docs.pytorch.org/docs/stable/generated/torch.Tensor.view.
    ↪ html#torch-tensor-view
    tensor = tensor.view((batch_size * depth), (height*width))

    gram = torch.mm(tensor, tensor.t())

    return gram

```

1.5 Putting it all Together

Now that we've written functions for extracting features and computing the gram matrix of a given convolutional layer; let's put all these pieces together! We'll extract our features from our images and calculate the gram matrices for each layer in our style representation.

```
[10]: # get content and style features only once before forming the target image
content_features = get_features(content, vgg)
style_features = get_features(style, vgg)

# calculate the gram matrices for each layer of our style representation
style_grams = {layer: gram_matrix(style_features[layer]) for layer in
    ↪ style_features}

# create a third "target" image and prep it for change
# it is a good idea to start off with the target as a copy of our *content*
    ↪ image
# then iteratively change its style
target = content.clone().requires_grad_(True).to(device)
```

1.6 Loss and Weights

Individual Layer Style Weights Below, you are given the option to weight the style representation at each relevant layer. It's suggested that you use a range between 0-1 to weight these layers. By weighting earlier layers (`conv1_1` and `conv2_1`) more, you can expect to get *larger* style artifacts in your resulting, target image. Should you choose to weight later layers, you'll get more emphasis on smaller features. This is because each layer is a different size and together they create a multi-scale style representation!

Content and Style Weight Just like in the paper, we define an alpha (`content_weight`) and a beta (`style_weight`). This ratio will affect how *stylized* your final image is. It's recommended that you leave the `content_weight = 1` and set the `style_weight` to achieve the ratio you want.

```
[11]: # weights for each style layer
# weighting earlier layers more will result in *larger* style artifacts
# notice we are excluding `conv4_2` our content representation
style_weights = {'conv1_1': 1.,
                 'conv2_1': 0.8,
                 'conv3_1': 0.5,
                 'conv4_1': 0.3,
                 'conv5_1': 0.1}

# you may choose to leave these as is
content_weight = 1 # alpha
style_weight = 1e6 # beta
```

1.7 Updating the Target & Calculating Losses

You'll decide on a number of steps for which to update your image, this is similar to the training loop that you've seen before, only we are changing our *target* image and nothing else about VGG19 or any other image. Therefore, the number of steps is really up to you to set! **I recommend**

using at least 2000 steps for good results. But, you may want to start out with fewer steps if you are just testing out different weight values or experimenting with different images.

Inside the iteration loop, you'll calculate the content and style losses and update your target image, accordingly.

Content Loss The content loss will be the mean squared difference between the target and content features at layer `conv4_2`. This can be calculated as follows:

```
content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])**2)
```

Style Loss The style loss is calculated in a similar way, only you have to iterate through a number of layers, specified by name in our dictionary `style_weights`. > You'll calculate the gram matrix for the target image, `target_gram` and style image `style_gram` at each of these layers and compare those gram matrices, calculating the `layer_style_loss`. > Later, you'll see that this value is normalized by the size of the layer.

Total Loss Finally, you'll create the total loss by adding up the style and content losses and weighting them with your specified alpha and beta!

Intermittently, we'll print out this loss; don't be alarmed if the loss is very large. It takes some time for an image's style to change and you should focus on the appearance of your target image rather than any loss value. Still, you should see that this loss decreases over some number of iterations.

TODO: Define content, style, and total losses.

```
[12]: # Needed this to see the progress bar...
      from tqdm.notebook import tqdm

      # for displaying the target image, intermittently
      # show_every = 50
      show_every = 400

      # iteration hyperparameters
      optimizer = optim.Adam([target], lr=0.003)
      # steps = 200
      steps = 2000 # decide how many iterations to update your image (5000)

      # for ii in range(1, steps+1):
      for ii in tqdm(range(1, steps+1), desc="Stylizing"):

          ## TODO: get the features from your target image
          ## Then calculate the content loss
          target_features = get_features(target, vgg)
          content_loss = torch.mean(
              (target_features["conv4_2"] - content_features["conv4_2"]) ** 2
          )

          # the style loss
```

```

# initialize the style loss to 0
style_loss = 0

# iterate through each style layer and add to the style loss
for layer in style_weights:
    # get the "target" style representation for the layer
    target_feature = target_features[layer]
    _, depth, height, width = target_feature.shape

    ## TODO: Calculate the target gram matrix
    target_gram = gram_matrix(target_feature)

    ## TODO: get the "style" style representation
    style_gram = style_grams[layer]
    ## TODO: Calculate the style loss for one layer, weighted appropriately
    layer_style_loss = style_weights[layer] * torch.mean(
        (target_gram - style_gram) ** 2
    )

    # add to the style loss
    style_loss += layer_style_loss / (depth * height * width)

## TODO: calculate the *total* loss
total_loss = content_weight * content_loss + style_weight * style_loss

## -- do not need to change code, below -- ##
# update your target image
optimizer.zero_grad()
total_loss.backward()
optimizer.step()

# Debug Again
if ii == 1:
    print("Style loss raw:", style_loss.item())
    Stylizing: 0%
7/2000 [00:08<39:43, 1.20s/it]

Style loss raw: 748.1528930664062

# Need to see the progress.... (Debug)
if ii % 10 == 0:
    print(
        f"Step {ii}/{steps} | "
        f"Total: {total_loss.item():.4f} | "
        f"Content: {content_loss.item():.4f} | "
        f"Style: {style_loss.item():.4f}"
    )

```

```

    )

    # display intermediate images and print the loss
    if ii % show_every == 0:
        print('Total loss: ', total_loss.item())
        plt.imshow(im_convert(target))
        plt.show()

    # Long story short, this is training the image, NOT the model
    # Or changing the input image ( the output of this)

```

Stylizing: 0% | 0/2000 [00:00<?, ?it/s]

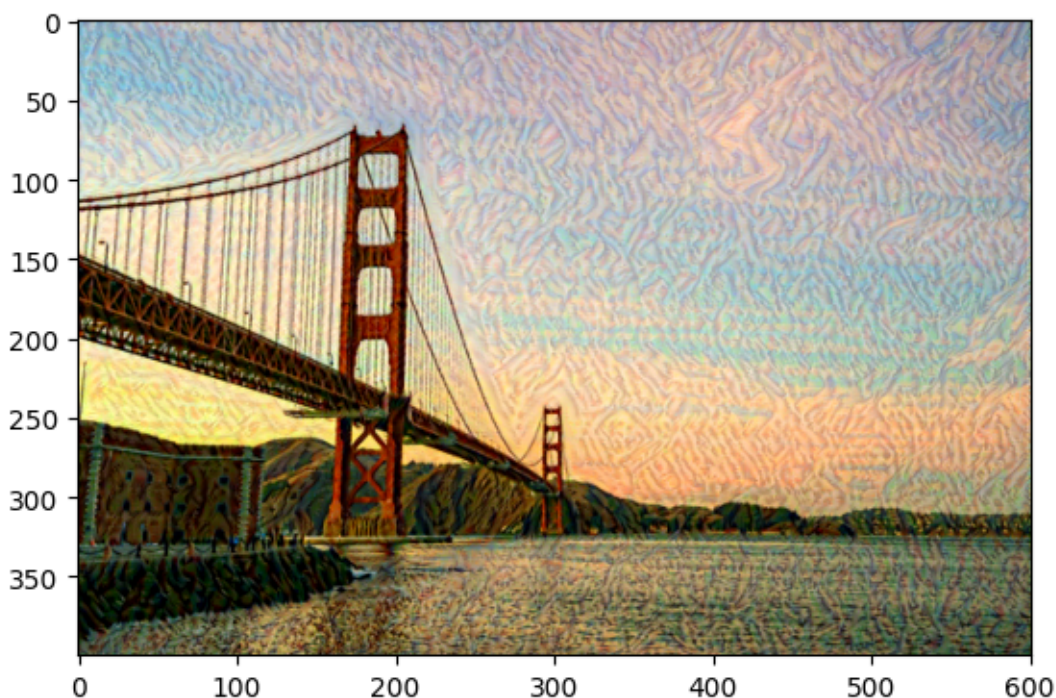
Style loss raw: 748.1528930664062

Step 10/2000	Total: 620272768.0000	Content: 1.5478	Style: 620.2728
Step 20/2000	Total: 495173920.0000	Content: 3.1062	Style: 495.1739
Step 30/2000	Total: 395239264.0000	Content: 4.0781	Style: 395.2393
Step 40/2000	Total: 315740192.0000	Content: 4.8489	Style: 315.7402
Step 50/2000	Total: 255178672.0000	Content: 5.5845	Style: 255.1787
Step 60/2000	Total: 210091056.0000	Content: 6.1803	Style: 210.0910
Step 70/2000	Total: 176433616.0000	Content: 6.6652	Style: 176.4336
Step 80/2000	Total: 150986000.0000	Content: 7.0816	Style: 150.9860
Step 90/2000	Total: 131336888.0000	Content: 7.4289	Style: 131.3369
Step 100/2000	Total: 115834576.0000	Content: 7.7177	Style: 115.8346
Step 110/2000	Total: 103325240.0000	Content: 7.9686	Style: 103.3252
Step 120/2000	Total: 93014912.0000	Content: 8.1921	Style: 93.0149
Step 130/2000	Total: 84378528.0000	Content: 8.3897	Style: 84.3785
Step 140/2000	Total: 77051288.0000	Content: 8.5659	Style: 77.0513
Step 150/2000	Total: 70756832.0000	Content: 8.7271	Style: 70.7568
Step 160/2000	Total: 65293764.0000	Content: 8.8741	Style: 65.2938
Step 170/2000	Total: 60513160.0000	Content: 9.0095	Style: 60.5132
Step 180/2000	Total: 56299328.0000	Content: 9.1382	Style: 56.2993
Step 190/2000	Total: 52558600.0000	Content: 9.2584	Style: 52.5586
Step 200/2000	Total: 49221612.0000	Content: 9.3694	Style: 49.2216
Step 210/2000	Total: 46230652.0000	Content: 9.4727	Style: 46.2306
Step 220/2000	Total: 43538340.0000	Content: 9.5684	Style: 43.5383
Step 230/2000	Total: 41107004.0000	Content: 9.6593	Style: 41.1070
Step 240/2000	Total: 38904764.0000	Content: 9.7451	Style: 38.9048
Step 250/2000	Total: 36902080.0000	Content: 9.8263	Style: 36.9021
Step 260/2000	Total: 35076032.0000	Content: 9.9032	Style: 35.0760
Step 270/2000	Total: 33407046.0000	Content: 9.9764	Style: 33.4070
Step 280/2000	Total: 31877680.0000	Content: 10.0471	Style: 31.8777
Step 290/2000	Total: 30472902.0000	Content: 10.1144	Style: 30.4729
Step 300/2000	Total: 29179486.0000	Content: 10.1796	Style: 29.1795
Step 310/2000	Total: 27986092.0000	Content: 10.2413	Style: 27.9861
Step 320/2000	Total: 26883214.0000	Content: 10.3000	Style: 26.8832
Step 330/2000	Total: 25862618.0000	Content: 10.3563	Style: 25.8626

```

Step 340/2000 | Total: 24915556.0000 | Content: 10.4108 | Style: 24.9155
Step 350/2000 | Total: 24035072.0000 | Content: 10.4631 | Style: 24.0351
Step 360/2000 | Total: 23215476.0000 | Content: 10.5131 | Style: 23.2155
Step 370/2000 | Total: 22451258.0000 | Content: 10.5613 | Style: 22.4512
Step 380/2000 | Total: 21737182.0000 | Content: 10.6077 | Style: 21.7372
Step 390/2000 | Total: 21068624.0000 | Content: 10.6530 | Style: 21.0686
Step 400/2000 | Total: 20441966.0000 | Content: 10.6968 | Style: 20.4420
Total loss: 20441966.0

```

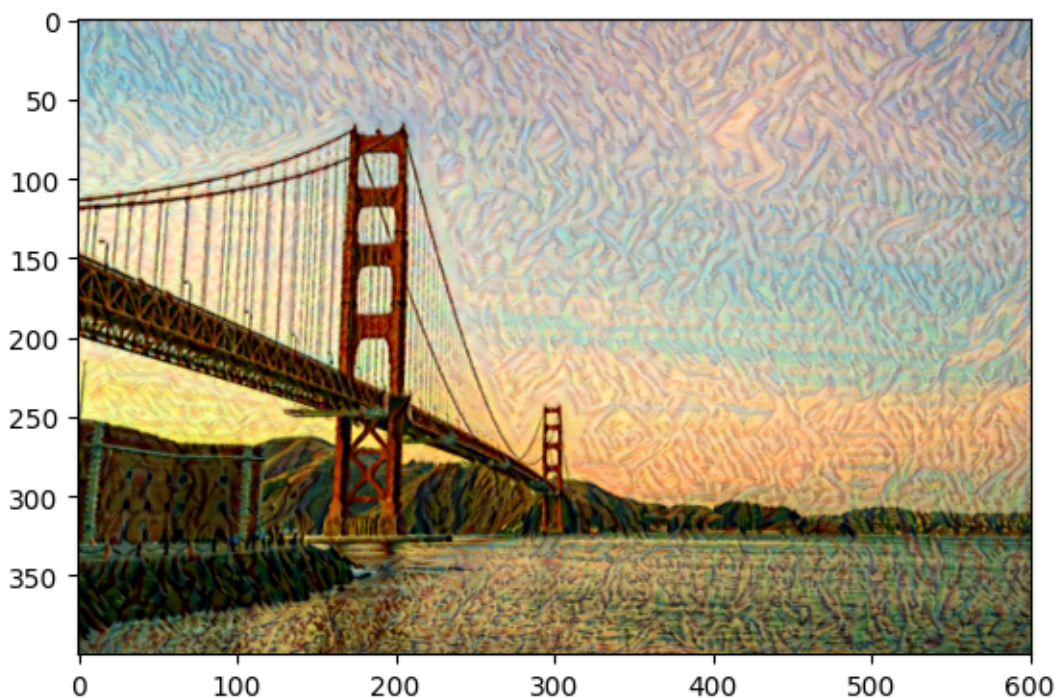


```

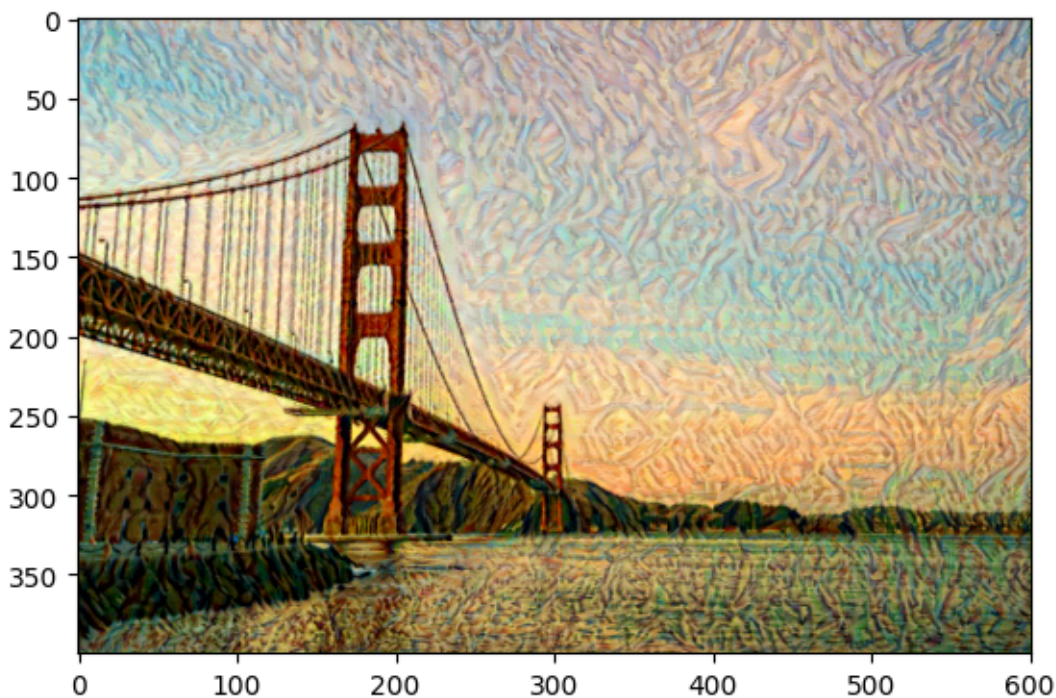
Step 410/2000 | Total: 19853408.0000 | Content: 10.7392 | Style: 19.8534
Step 420/2000 | Total: 19299870.0000 | Content: 10.7807 | Style: 19.2999
Step 430/2000 | Total: 18778412.0000 | Content: 10.8206 | Style: 18.7784
Step 440/2000 | Total: 18286578.0000 | Content: 10.8594 | Style: 18.2866
Step 450/2000 | Total: 17821422.0000 | Content: 10.8970 | Style: 17.8214
Step 460/2000 | Total: 17380790.0000 | Content: 10.9334 | Style: 17.3808
Step 470/2000 | Total: 16963098.0000 | Content: 10.9686 | Style: 16.9631
Step 480/2000 | Total: 16566478.0000 | Content: 11.0028 | Style: 16.5665
Step 490/2000 | Total: 16189258.0000 | Content: 11.0371 | Style: 16.1892
Step 500/2000 | Total: 15830082.0000 | Content: 11.0705 | Style: 15.8301
Step 510/2000 | Total: 15487589.0000 | Content: 11.1029 | Style: 15.4876
Step 520/2000 | Total: 15160728.0000 | Content: 11.1344 | Style: 15.1607
Step 530/2000 | Total: 14848329.0000 | Content: 11.1647 | Style: 14.8483
Step 540/2000 | Total: 14549401.0000 | Content: 11.1942 | Style: 14.5494
Step 550/2000 | Total: 14263180.0000 | Content: 11.2225 | Style: 14.2632
Step 560/2000 | Total: 13988818.0000 | Content: 11.2499 | Style: 13.9888

```


Step 570/2000	Total: 13725440.0000	Content: 11.2769	Style: 13.7254
Step 580/2000	Total: 13472378.0000	Content: 11.3031	Style: 13.4724
Step 590/2000	Total: 13229061.0000	Content: 11.3286	Style: 13.2290
Step 600/2000	Total: 12994734.0000	Content: 11.3538	Style: 12.9947
Step 610/2000	Total: 12768981.0000	Content: 11.3786	Style: 12.7690
Step 620/2000	Total: 12551318.0000	Content: 11.4027	Style: 12.5513
Step 630/2000	Total: 12341382.0000	Content: 11.4269	Style: 12.3414
Step 640/2000	Total: 12138657.0000	Content: 11.4507	Style: 12.1386
Step 650/2000	Total: 11942745.0000	Content: 11.4744	Style: 11.9427
Step 660/2000	Total: 11753221.0000	Content: 11.4975	Style: 11.7532
Step 670/2000	Total: 11569786.0000	Content: 11.5202	Style: 11.5698
Step 680/2000	Total: 11392021.0000	Content: 11.5422	Style: 11.3920
Step 690/2000	Total: 11219676.0000	Content: 11.5636	Style: 11.2197
Step 700/2000	Total: 11052446.0000	Content: 11.5846	Style: 11.0524
Step 710/2000	Total: 10890109.0000	Content: 11.6054	Style: 10.8901
Step 720/2000	Total: 10732392.0000	Content: 11.6256	Style: 10.7324
Step 730/2000	Total: 10579084.0000	Content: 11.6451	Style: 10.5791
Step 740/2000	Total: 10429960.0000	Content: 11.6643	Style: 10.4299
Step 750/2000	Total: 10284813.0000	Content: 11.6833	Style: 10.2848
Step 760/2000	Total: 10143560.0000	Content: 11.7017	Style: 10.1435
Step 770/2000	Total: 10006061.0000	Content: 11.7198	Style: 10.0060
Step 780/2000	Total: 9872182.0000	Content: 11.7374	Style: 9.8722
Step 790/2000	Total: 9741758.0000	Content: 11.7550	Style: 9.7417
Step 800/2000	Total: 9614601.0000	Content: 11.7728	Style: 9.6146
Total loss: 9614601.0			

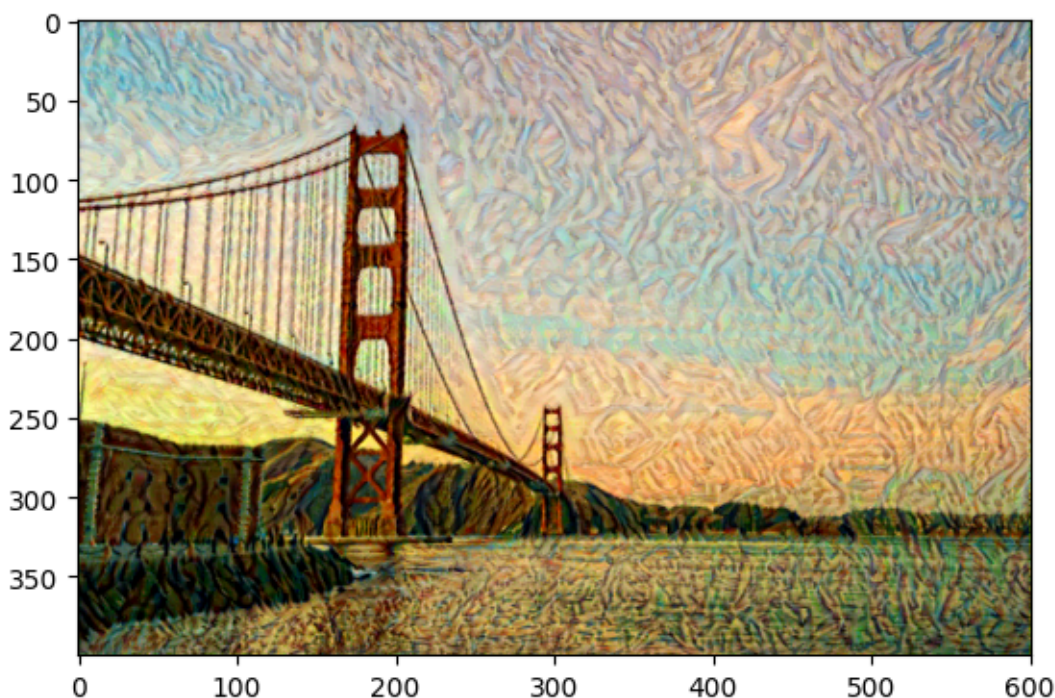


Step 810/2000	Total: 9490484.0000	Content: 11.7903	Style: 9.4905
Step 820/2000	Total: 9369335.0000	Content: 11.8076	Style: 9.3693
Step 830/2000	Total: 9251037.0000	Content: 11.8249	Style: 9.2510
Step 840/2000	Total: 9135534.0000	Content: 11.8420	Style: 9.1355
Step 850/2000	Total: 9022661.0000	Content: 11.8588	Style: 9.0226
Step 860/2000	Total: 8912313.0000	Content: 11.8755	Style: 8.9123
Step 870/2000	Total: 8804379.0000	Content: 11.8918	Style: 8.8044
Step 880/2000	Total: 8698796.0000	Content: 11.9078	Style: 8.6988
Step 890/2000	Total: 8595495.0000	Content: 11.9235	Style: 8.5955
Step 900/2000	Total: 8494342.0000	Content: 11.9388	Style: 8.4943
Step 910/2000	Total: 8395341.0000	Content: 11.9537	Style: 8.3953
Step 920/2000	Total: 8298387.0000	Content: 11.9687	Style: 8.2984
Step 930/2000	Total: 8203373.5000	Content: 11.9836	Style: 8.2034
Step 940/2000	Total: 8110192.0000	Content: 11.9981	Style: 8.1102
Step 950/2000	Total: 8018840.5000	Content: 12.0125	Style: 8.0188
Step 960/2000	Total: 7929293.0000	Content: 12.0266	Style: 7.9293
Step 970/2000	Total: 7841462.0000	Content: 12.0405	Style: 7.8415
Step 980/2000	Total: 7755289.5000	Content: 12.0542	Style: 7.7553
Step 990/2000	Total: 7670756.5000	Content: 12.0679	Style: 7.6707
Step 1000/2000	Total: 7587777.0000	Content: 12.0814	Style: 7.5878
Step 1010/2000	Total: 7506319.5000	Content: 12.0948	Style: 7.5063
Step 1020/2000	Total: 7426320.5000	Content: 12.1083	Style: 7.4263
Step 1030/2000	Total: 7347763.5000	Content: 12.1217	Style: 7.3478
Step 1040/2000	Total: 7270616.0000	Content: 12.1347	Style: 7.2706
Step 1050/2000	Total: 7194833.5000	Content: 12.1476	Style: 7.1948
Step 1060/2000	Total: 7120352.5000	Content: 12.1602	Style: 7.1203
Step 1070/2000	Total: 7047143.5000	Content: 12.1726	Style: 7.0471
Step 1080/2000	Total: 6975189.0000	Content: 12.1849	Style: 6.9752
Step 1090/2000	Total: 6904401.0000	Content: 12.1970	Style: 6.9044
Step 1100/2000	Total: 6834738.0000	Content: 12.2089	Style: 6.8347
Step 1110/2000	Total: 6766194.5000	Content: 12.2210	Style: 6.7662
Step 1120/2000	Total: 6698746.5000	Content: 12.2329	Style: 6.6987
Step 1130/2000	Total: 6632383.5000	Content: 12.2447	Style: 6.6324
Step 1140/2000	Total: 6567051.5000	Content: 12.2567	Style: 6.5670
Step 1150/2000	Total: 6502732.5000	Content: 12.2685	Style: 6.5027
Step 1160/2000	Total: 6439404.5000	Content: 12.2802	Style: 6.4394
Step 1170/2000	Total: 6377027.0000	Content: 12.2916	Style: 6.3770
Step 1180/2000	Total: 6315606.5000	Content: 12.3030	Style: 6.3156
Step 1190/2000	Total: 6255089.5000	Content: 12.3141	Style: 6.2551
Step 1200/2000	Total: 6195462.5000	Content: 12.3252	Style: 6.1954
Total loss: 6195462.5			



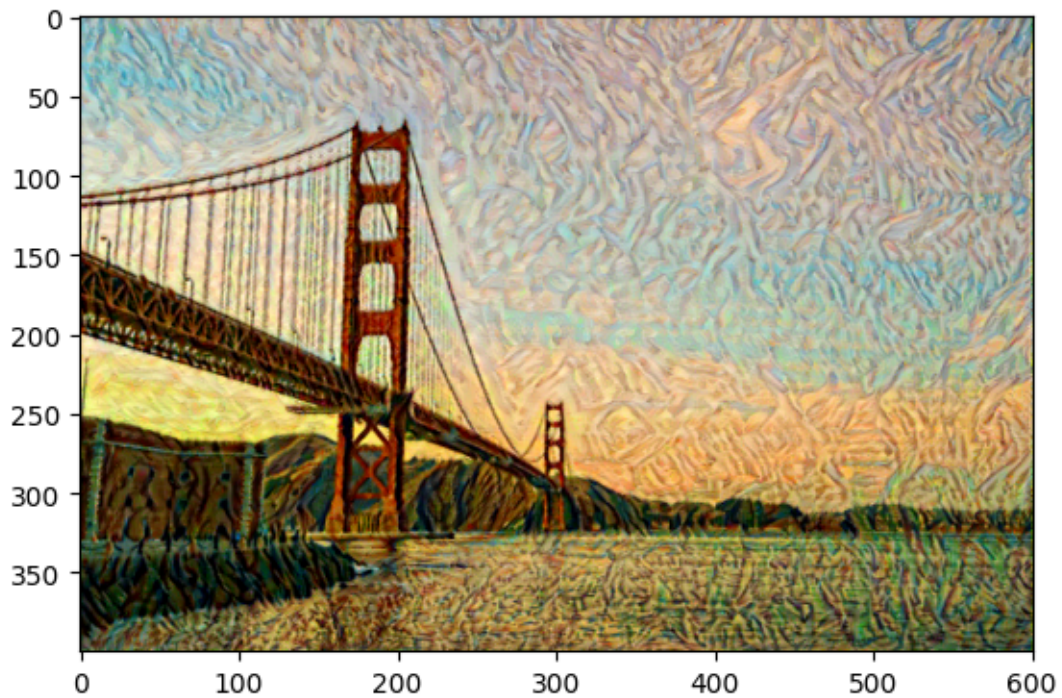
Step 1210/2000	Total: 6136703.5000	Content: 12.3361	Style: 6.1367
Step 1220/2000	Total: 6078813.0000	Content: 12.3470	Style: 6.0788
Step 1230/2000	Total: 6021747.5000	Content: 12.3577	Style: 6.0217
Step 1240/2000	Total: 5965486.0000	Content: 12.3684	Style: 5.9655
Step 1250/2000	Total: 5910003.5000	Content: 12.3788	Style: 5.9100
Step 1260/2000	Total: 5855279.5000	Content: 12.3892	Style: 5.8553
Step 1270/2000	Total: 5801310.0000	Content: 12.3993	Style: 5.8013
Step 1280/2000	Total: 5748044.5000	Content: 12.4096	Style: 5.7480
Step 1290/2000	Total: 5695476.0000	Content: 12.4197	Style: 5.6955
Step 1300/2000	Total: 5643600.5000	Content: 12.4297	Style: 5.6436
Step 1310/2000	Total: 5592427.5000	Content: 12.4397	Style: 5.5924
Step 1320/2000	Total: 5541925.0000	Content: 12.4493	Style: 5.5419
Step 1330/2000	Total: 5492076.0000	Content: 12.4591	Style: 5.4921
Step 1340/2000	Total: 5442873.0000	Content: 12.4690	Style: 5.4429
Step 1350/2000	Total: 5394301.5000	Content: 12.4786	Style: 5.3943
Step 1360/2000	Total: 5346349.5000	Content: 12.4882	Style: 5.3463
Step 1370/2000	Total: 5299007.5000	Content: 12.4976	Style: 5.2990
Step 1380/2000	Total: 5252257.5000	Content: 12.5070	Style: 5.2522
Step 1390/2000	Total: 5206071.5000	Content: 12.5163	Style: 5.2061
Step 1400/2000	Total: 5160442.0000	Content: 12.5256	Style: 5.1604
Step 1410/2000	Total: 5115368.5000	Content: 12.5351	Style: 5.1154
Step 1420/2000	Total: 5070830.0000	Content: 12.5443	Style: 5.0708
Step 1430/2000	Total: 5026814.0000	Content: 12.5534	Style: 5.0268
Step 1440/2000	Total: 4983295.5000	Content: 12.5622	Style: 4.9833

Step 1450/2000	Total: 4940255.5000	Content: 12.5708	Style: 4.9402
Step 1460/2000	Total: 4897708.0000	Content: 12.5797	Style: 4.8977
Step 1470/2000	Total: 4855657.5000	Content: 12.5885	Style: 4.8556
Step 1480/2000	Total: 4814079.5000	Content: 12.5972	Style: 4.8141
Step 1490/2000	Total: 4772961.5000	Content: 12.6061	Style: 4.7729
Step 1500/2000	Total: 4732298.5000	Content: 12.6148	Style: 4.7323
Step 1510/2000	Total: 4692089.0000	Content: 12.6235	Style: 4.6921
Step 1520/2000	Total: 4652322.0000	Content: 12.6322	Style: 4.6523
Step 1530/2000	Total: 4613003.5000	Content: 12.6409	Style: 4.6130
Step 1540/2000	Total: 4574130.0000	Content: 12.6496	Style: 4.5741
Step 1550/2000	Total: 4535659.0000	Content: 12.6585	Style: 4.5356
Step 1560/2000	Total: 4497589.5000	Content: 12.6672	Style: 4.4976
Step 1570/2000	Total: 4459928.5000	Content: 12.6758	Style: 4.4599
Step 1580/2000	Total: 4422667.0000	Content: 12.6844	Style: 4.4227
Step 1590/2000	Total: 4385798.5000	Content: 12.6928	Style: 4.3858
Step 1600/2000	Total: 4349316.5000	Content: 12.7012	Style: 4.3493
Total loss: 4349316.5			



Step 1610/2000	Total: 4313204.5000	Content: 12.7094	Style: 4.3132
Step 1620/2000	Total: 4277453.0000	Content: 12.7177	Style: 4.2774
Step 1630/2000	Total: 4242056.5000	Content: 12.7260	Style: 4.2420
Step 1640/2000	Total: 4207006.5000	Content: 12.7343	Style: 4.2070
Step 1650/2000	Total: 4172302.0000	Content: 12.7424	Style: 4.1723
Step 1660/2000	Total: 4137945.0000	Content: 12.7504	Style: 4.1379
Step 1670/2000	Total: 4103925.0000	Content: 12.7583	Style: 4.1039

Step 1680/2000	Total: 4070240.2500	Content: 12.7660	Style: 4.0702
Step 1690/2000	Total: 4036878.0000	Content: 12.7735	Style: 4.0369
Step 1700/2000	Total: 4003843.2500	Content: 12.7810	Style: 4.0038
Step 1710/2000	Total: 3971132.7500	Content: 12.7884	Style: 3.9711
Step 1720/2000	Total: 3938749.7500	Content: 12.7958	Style: 3.9387
Step 1730/2000	Total: 3906676.0000	Content: 12.8032	Style: 3.9067
Step 1740/2000	Total: 3874904.2500	Content: 12.8108	Style: 3.8749
Step 1750/2000	Total: 3843435.2500	Content: 12.8181	Style: 3.8434
Step 1760/2000	Total: 3812270.7500	Content: 12.8253	Style: 3.8123
Step 1770/2000	Total: 3781407.2500	Content: 12.8325	Style: 3.7814
Step 1780/2000	Total: 3750840.2500	Content: 12.8395	Style: 3.7508
Step 1790/2000	Total: 3720562.2500	Content: 12.8464	Style: 3.7205
Step 1800/2000	Total: 3690569.2500	Content: 12.8535	Style: 3.6906
Step 1810/2000	Total: 3660862.7500	Content: 12.8603	Style: 3.6609
Step 1820/2000	Total: 3631440.5000	Content: 12.8671	Style: 3.6314
Step 1830/2000	Total: 3602288.7500	Content: 12.8739	Style: 3.6023
Step 1840/2000	Total: 3573411.7500	Content: 12.8806	Style: 3.5734
Step 1850/2000	Total: 3544804.0000	Content: 12.8872	Style: 3.5448
Step 1860/2000	Total: 3516465.5000	Content: 12.8938	Style: 3.5165
Step 1870/2000	Total: 3488393.0000	Content: 12.9003	Style: 3.4884
Step 1880/2000	Total: 3460579.7500	Content: 12.9068	Style: 3.4606
Step 1890/2000	Total: 3433009.5000	Content: 12.9132	Style: 3.4330
Step 1900/2000	Total: 3405677.0000	Content: 12.9197	Style: 3.4057
Step 1910/2000	Total: 3378590.0000	Content: 12.9260	Style: 3.3786
Step 1920/2000	Total: 3351757.2500	Content: 12.9324	Style: 3.3517
Step 1930/2000	Total: 3325172.2500	Content: 12.9388	Style: 3.3252
Step 1940/2000	Total: 3298832.7500	Content: 12.9453	Style: 3.2988
Step 1950/2000	Total: 3272724.2500	Content: 12.9516	Style: 3.2727
Step 1960/2000	Total: 3246845.5000	Content: 12.9580	Style: 3.2468
Step 1970/2000	Total: 3221197.5000	Content: 12.9643	Style: 3.2212
Step 1980/2000	Total: 3195782.2500	Content: 12.9705	Style: 3.1958
Step 1990/2000	Total: 3170589.0000	Content: 12.9765	Style: 3.1706
Step 2000/2000	Total: 3145624.7500	Content: 12.9827	Style: 3.1456
Total loss: 3145624.75			



1.8 Display the Target Image

```
[13]: # display content and final, target image
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))
ax1.imshow(im_convert(content))
ax2.imshow(im_convert(target))
```

```
[13]: <matplotlib.image.AxesImage at 0x7f3abfd37250>
```

